



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Syntax Validator

Curso: Base de Datos II

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

Arocutipa Arocutipa, Gian Franco (2023076790)

Soto Oquendo Cristian Gabriel (2026086510)

Tacna – Perú

2026

Validador de Sintaxis SQL/NoSQL Informe de Factibilidad

Versión 1.0



CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	GA, CS			27/03/2026	Versión Original
2.0	GA, CS			04/07/2026	Corrección del Informe

ÍNDICE GENERAL

1.	Descripción del Proyecto	3
2.	Riesgos	3
3.	Análisis de la Situación actual	3
4.	Estudio de Factibilidad	3
4.1	Factibilidad Técnica	4
4.2	Factibilidad económica	4
4.3	Factibilidad Operativa	4
4.4	Factibilidad Legal	4
4.5	Factibilidad Social	5
4.6	Factibilidad Ambiental	5
5.	Análisis Financiero	5
6.	Conclusiones	5



Informe de Factibilidad

1. Descripción del Proyecto

1.1 Nombre del proyecto

Implementación del Validador de Sintaxis SQL/NoSQL, una plataforma que analiza y valida la sintaxis de consultas SQL multimotor (SQL ANSI, MySQL, MariaDB, PostgreSQL, SQL Server, Oracle, SQLite) y comandos MongoDB, compuesta por una aplicación web, una Skill/API pública REST y un CLI instalable para Linux, desplegada actualmente en Railway.

1.2 Duración del proyecto

Se estima una duración de 3 meses, estructurada en 5 fases de desarrollo iterativo para garantizar un despliegue estable de la web, la Skill/API y el CLI.

1.3 Descripción

El proyecto consiste en el desarrollo de un motor de análisis léxico y sintáctico propio (Lexer, SQLParser, MongoParser) capaz de detectar errores de sintaxis en consultas SQL y comandos MongoDB sin necesidad de ejecutarlas contra un motor de base de datos real. A diferencia de librerías genéricas de parsing SQL, el sistema reimplementa su propio tokenizador y analizador sintáctico, con detección automática del motor SQL más probable (MySQL, PostgreSQL, SQL Server, Oracle, SQLite, MariaDB) mediante un sistema de puntuación por palabras clave, tipos de datos y sintaxis específica de cada motor, generando errores con línea, columna, mensaje y sugerencia de corrección.

1.4 Objetivos

1.4.1 Objetivo general

Desarrollar una plataforma multimotor de validación sintáctica de consultas SQL y NoSQL, accesible mediante una interfaz web, una Skill/API pública y un CLI, con un panel administrativo que registre y audite el historial de validaciones y accesos de los usuarios.

1.4.2 Objetivos Específicos

- *Desarrollar un lexer y parser propios para SQL multimotor (SQL ANSI, MySQL, MariaDB, PostgreSQL, SQL Server, Oracle, SQLite) que detecten errores de sintaxis con línea, columna y sugerencia de corrección.*
- *Implementar un motor de detección automática del dialecto SQL basado en palabras clave, tipos de datos y patrones de sintaxis propios de cada motor, con nivel de confianza y lista de motores compatibles.*
- *Implementar un parser de comandos MongoDB (find, insertOne, updateOne, aggregate) que valide la sintaxis de operadores y estructura de los documentos JSON.*
- *Diseñar y programar una Skill/API pública REST versionada (/api/v1) que permita a agentes de IA, CLIs y editores de código consumir el validador sin reimplementar la gramática.*
- *Desarrollar un CLI instalable en Linux (sqlcheck), empaquetado en formatos .deb, .rpm y .tar.gz, para validar consultas desde terminal, scripts o pipelines.*
- *Implementar un sistema de autenticación basado en JWT y roles (user/admin/superadmin) que garantice que el panel administrativo y los registros de auditoría solo sean accesibles por personal autorizado.*

2. Riesgos

Falsos positivos/negativos en la detección del motor SQL: el sistema de puntuación por palabras clave puede identificar incorrectamente el motor de una consulta ambigua o compatible con varios dialectos.

Cobertura incompleta de la gramática SQL: al ser un parser propio y no una librería estándar madura, ciertas construcciones SQL avanzadas o poco comunes podrían no estar cubiertas.

Degradación de rendimiento en archivos grandes: la validación de archivos SQL/NoSQL muy grandes por streaming podría afectar el tiempo de respuesta bajo carga concurrente.

Riesgo	Descripción	Probabilidad	Impacto
Detección de motor incorrecta	El sistema de puntuación por palabras clave detecta un motor SQL distinto al esperado en consultas ambiguas.	Media	Medio

Cobertura incompleta de gramática	Construcciones SQL avanzadas o poco comunes no reconocidas por el parser propio.	Media	Alto
Rendimiento en archivos grandes	Lentitud al validar archivos SQL/NoSQL muy extensos o con mucha concurrencia.	Media	Medio
Disponibilidad	Ausencia temporal de un integrante clave del equipo.	Baja	Alto

3. Análisis de la Situación actual

3.1 Planteamiento del problema

Los desarrolladores, administradores de bases de datos y estudiantes que trabajan con múltiples motores SQL y MongoDB enfrentan una deficiencia crítica en la detección temprana de errores de sintaxis, provocada por la falta de una herramienta centralizada de validación multimotor, dividida en tres causas raíz que alimentan un ciclo de ineficiencia:

- *La ausencia de una validación previa a la ejecución permite que consultas con errores de sintaxis se ejecuten directamente contra el motor de base de datos, consumiendo recursos del servidor de forma innecesaria.*
- *Cada motor SQL (MySQL, PostgreSQL, SQL Server, Oracle, SQLite) genera mensajes de error con formato y nivel de detalle distintos, lo que dificulta a un desarrollador que trabaja con varios motores interpretar y corregir errores de forma consistente.*
- *No existe una herramienta única que permita validar consultas SQL de distintos dialectos y comandos MongoDB en un mismo lugar, sin necesidad de tener instalado cada motor o de integrar múltiples librerías de parsing.*
- **Árbol de problemas:**



Diagrama disponible en el repositorio del proyecto (README.md), junto con los diagramas de arquitectura, secuencia y despliegue del Validador de Sintaxis SQL/NoSQL.

El Problema Central

Errores de sintaxis SQL/NoSQL no detectados antes de su ejecución, por la falta de una herramienta multimotor centralizada de validación previa.

Causas Raíz

El diagrama identifica seis áreas críticas donde los desarrolladores y estudiantes fallan actualmente:

- ***Detección tardía de errores: los errores de sintaxis se descubren recién al ejecutar la consulta contra el motor de base de datos real.***
- ***Falta de trazabilidad de las validaciones: no existe un historial centralizado de qué consultas se validaron, cuándo y con qué resultado.***
- ***Herramientas dispersas por motor: cada IDE o cliente de base de datos (MySQL Workbench, pgAdmin, SSMS, MongoDB Compass) valida solo su propio dialecto, sin integración entre sí.***
- ***Mensajes de error poco descriptivos: los mensajes nativos de cada DBMS suelen ser crípticos y difíciles de interpretar para usuarios con menor experiencia.***
- ***Ausencia de sugerencias de corrección: los errores, cuando se detectan, no siempre indican cómo corregirlos, dificultando el aprendizaje y la depuración.***
- ***Falta de integraciones externas: no existía una API pública ni un CLI que permitiera a agentes de IA, editores o pipelines de CI/CD validar consultas sin reimplementar la gramática SQL.***

Efectos y Consecuencias

Estas fallas desencadenan una reacción en cadena que termina en:

- **Inmediatos:** consultas con errores de sintaxis (por ejemplo, un **DELETE** o **UPDATE** sin **WHERE**) llegan a ejecutarse contra una base de datos real.
- **A medio plazo:** incidentes operativos, retrabajo por errores detectados tarde y pérdida de tiempo de desarrollo en depuración manual.
- **Finales:** pérdida o corrupción de datos en entornos de producción, y mayor costo de remediación frente a detectar el error antes de la ejecución.

3.2 Consideraciones de hardware y software

Se utiliza Node.js 18+ con Express para el backend de la web principal y de la Skill/API pública, y TypeScript para el CLI (sqlcheck). Para el despliegue en la nube se usa Railway como entorno de producción activo, con PostgreSQL gestionado como base de datos; adicionalmente se documentan plantillas Terraform opcionales para AWS (ECS Fargate) y Azure (Container Apps) como alternativas de despliegue reproducible, no activas en la producción actual.

Categoría	Elemento	Descripción / Especificación
Hardware	Computadoras de desarrollo	Laptops con mínimo 8 GB RAM, procesador Intel Core i5 o superior, para codificación, pruebas locales y ejecución de contenedores Docker.
	Dispositivos para pruebas (navegador / CLI)	Equipos usados para validar la interfaz web, el editor Monaco y el CLI sqlcheck en distintos entornos.
	Red / Internet del equipo	Conectividad necesaria para el desarrollo colaborativo, despliegue en Railway y ejecución de la pipeline CI/CD.
Software	Node.js 18+ / Express	Lenguaje y framework principal del backend. Procesa la lógica de

		validación, autenticación y auditoría.
	Monaco Editor	Editor de código embebido en el frontend web: resaltado de sintaxis, numeración de líneas y marcado de errores.
	TypeScript (CLI sqlcheck)	Lenguaje del CLI, empaquetado como binarios .deb, .rpm y .tar.gz para Linux.
	PostgreSQL (Railway)	Sistema gestor de base de datos relacional gestionado, utilizado para usuarios, administradores, sesiones y auditoría. Garantiza integridad y concurrencia de datos.
	Docker	Plataforma de contenedores utilizada para empaquetar la web principal y la Skill/API, garantizando que el entorno de desarrollo sea idéntico al de producción.
	Git y GitHub (Control de versiones + CI/CD)	Sistema de control de versiones distribuido y pipeline de GitHub Actions (Jest, Stryker, Cucumber, Semgrep, Snyk) para integración continua.
Infraestructura Cloud	Railway	Plataforma de despliegue activa en producción para la web principal, la Skill/API y la base de datos PostgreSQL.
	Terraform (AWS / Azure, opcional)	Infraestructura como código documentada como alternativa de despliegue reproducible (ECS Fargate en AWS, Container Apps en Azure), no activa en la producción actual.

4. Estudio de Factibilidad

4.1 Factibilidad Técnica

El proyecto es altamente viable. El stack (Node.js/Express) es maduro y ya se encuentra desplegado en producción en Railway, lo que reduce el riesgo técnico de

la infraestructura. El motor de validación propio (Lexer/Parser) ya está implementado y cubierto con pruebas unitarias, de mutación (Stryker) y BDD (Cucumber).

Categoría	Elemento	Descripción / Especificación
Hardware	Computadoras de desarrollo	Laptops con mínimo 8 GB RAM, procesador Intel Core i5 o superior, para codificación, pruebas locales y ejecución de contenedores Docker.
	Red / Internet del equipo	Conectividad necesaria para el desarrollo colaborativo, despliegue en la nube y ejecución de la pipeline CI/CD.
Software	Node.js 18+ (Express) + Monaco Editor	Backend de validación y editor de código interactivo.
Infraestructura Cloud	Railway (+ Terraform opcional)	Despliegue reproducible de la web y la Skill/API, con alternativas documentadas en AWS/Azure.

4.2 Factibilidad Económica

4.2.1 Costos Generales

Comprende los equipos de cómputo de los integrantes del equipo y los insumos de escritorio necesarios para la documentación formal del proyecto.

Ítem	Descripción	Costo Estimado (S/)
Equipos de cómputo	2 laptops de los integrantes del equipo (recursos propios). Cap. mínima: 8 GB RAM, i5, 256 GB SSD. No representa costo adicional al proyecto.	S/ 0.00 (recurso propio)
Insumos de escritorio	Materiales de papelería, folder, anillados y útiles para documentación formal del proyecto.	S/ 80.00

<i>GitHub Team (Control de versiones)</i>	<i>Plan GitHub Free: repositorios privados ilimitados para equipos pequeños, suficiente para el control de versiones del proyecto. \$0 USD.</i>	<i>S/ 0.00</i>
<i>Figma Free (Diseño UI/UX)</i>	<i>Plan Figma Free: diseño de interfaces y prototipado del frontend web. Plan gratuito con hasta 3 proyectos activos. \$0 USD.</i>	<i>S/ 0.00</i>
<i>Notion Free (Gestión de tareas)</i>	<i>Plan Notion Free: gestión de tareas, documentación técnica y backlog del proyecto. Plan gratuito ilimitado para equipos pequeños. \$0 USD.</i>	<i>S/ 0.00</i>
	<i>SUBTOTAL COSTOS GENERALES</i>	<i>S/ 80.00</i>

4.2.2 Costos operativos durante el desarrollo

Corresponde a los gastos recurrentes generados durante los tres meses de ejecución activa del proyecto: energía eléctrica, conectividad a internet y comunicaciones del equipo (proyecto de desarrollo 100% remoto).

<i>Concepto</i>	<i>Descripción</i>	<i>Costo Mensual (S/)</i>	<i>Total 3 meses (S/)</i>
<i>Energía Eléctrica</i>	<i>Consumo estimado de 2 laptops (8h/día, 6 días/semana) + iluminación del espacio de trabajo compartido.</i>	<i>S/ 60.00</i>	<i>S/ 180.00</i>
<i>Servicio de Internet</i>	<i>Plan de internet dedicado para desarrollo: acceso a repositorios, despliegue en Railway y videollamadas de coordinación de equipo.</i>	<i>S/ 99.00</i>	<i>S/ 297.00</i>
<i>Comunicaciones del equipo</i>	<i>Llamadas y datos móviles adicionales para coordinación remota del equipo.</i>	<i>S/ 20.00</i>	<i>S/ 60.00</i>
	<i>SUBTOTAL COSTOS OPERATIVOS</i>	<i>S/ 179.00</i>	<i>S/ 537.00</i>



Post-lanzamiento, los costos operativos de desarrollo (energía, internet dedicado, comunicaciones del equipo) cesan; el único costo recurrente que continúa indefinidamente es la infraestructura cloud (Railway), detallada en la sección de Costos del Ambiente.

4.2.3 Costos del ambiente

Incluye la infraestructura activa en Railway (web principal, Skill/API y base de datos PostgreSQL), el repositorio de código y la pipeline de CI/CD en GitHub Actions. El costo de cómputo en Railway es variable según el uso (facturación por consumo), por lo que el costo real puede ser menor al aquí estimado.

Servicio	Detalle	Costo/mes (S/)	Total 3 meses (S/)
GitHub Team (Repositorios + CI/CD)	Plan GitHub Team: repositorios privados ilimitados, GitHub Actions para pipelines CI/CD (Jest, Stryker, Cucumber, Semgrep, Snyk), protección de ramas y revisión de código. \$4 USD/usuario/mes × 2 usuarios.	S/ 30.00	S/ 90.00
Railway — Web principal + Skill/API	Hosting de la aplicación web y la Skill/API pública, facturación por consumo (CPU/RAM/red). Estimado bajo tráfico académico moderado. \$8 USD/mes.	S/ 30.00	S/ 90.00
Railway — PostgreSQL	Base de datos gestionada para usuarios, sesiones, administradores y auditoría. \$5 USD/mes.	S/ 18.75	S/ 56.25



Dominio / certificados (incluidos en Railway)	Subdominio *.up.railway.app con certificado TLS incluido, sin costo adicional.	S/ 0.00	S/ 0.00
Azure Storage Account — backend remoto de Terraform (.tfstate)	SUBTOTAL COSTOS DEL AMBIENTE	S/ 78.75	S/ 236.25
	SUBTOTAL COSTOS DEL AMBIENTE	S/ 120.40	S/ 361.20

4.2.4 Costos de personal

Valoración de las horas-hombre dedicadas al diseño del motor de validación SQL/NoSQL, desarrollo de la Skill/API, el CLI y la redacción de pruebas unitarias, de mutación y BDD.

Rol	Nombre	Horas/mes	Valor Hora (S/)	Total/mes (S/)	Total 3 meses (S/)
<i>Jefe de Proyecto / Dev. Backend</i>	<i>Gian Franco Arocutipa Arocutipa</i>	<i>90 h/mes</i>	<i>S/ 25.00</i>	<i>S/ 2,250.00</i>	<i>S/ 6,750.00</i>
<i>Analista Programador / Dev. Frontend</i>	<i>Fabrizio Salvador Elias Perez Peralta</i>	<i>90 h/mes</i>	<i>S/ 20.00</i>	<i>S/ 1,800.00</i>	<i>S/ 5,400.00</i>
			SUBTOTAL PERSONAL	S/ 4,050.00	S/ 12,150.00

4.2.5 Costos totales del desarrollo del sistema

Estimación final basada en el cronograma de ejecución del proyecto, incluyendo la infraestructura activa en Railway.

N°	Concepto de Gasto	Monto (S/)	Acumulado (S/)
1	Personal (Jefe de Proyecto + 1 Analista Programador × 3 meses)	S/ 12,150.00	S/ 12,150.00
2	Hardware (disco externo de respaldo y accesorios menores)	S/ 100.00	S/ 12,250.00
3	Infraestructura y Servicios Cloud vía Railway (3 meses)	S/ 236.25	S/ 12,486.25
4	Costos Operativos (Energía, Internet, Comunicaciones × 3 meses)	S/ 537.00	S/ 13,023.25
5	Costos Generales (Insumos + Herramientas de Software)	S/ 80.00	S/ 13,103.25
	TOTAL LÍNEA BASE		S/ 13,103.25
6	Reserva de Contingencia — 10% del total (riesgos técnicos identificados)	S/ 1,310.33	S/ 14,413.58
7	Reserva de Gestión — 5% del total (imprevistos administrativos)	S/ 655.16	S/ 15,068.74
	TOTAL PRESUPUESTO DEL PROYECTO		S/ 15,068.74

4.3 Factibilidad Operativa



El sistema es operativamente viable dado que se integra directamente en el flujo de trabajo de desarrollo existente (editor de código, terminal y pipeline CI/CD) sin requerir una reestructuración organizacional. Los perfiles de usuario (Administrador, Usuario) cuentan con interfaces diseñadas específicamente para sus necesidades:

- El Administrador dispondrá de un panel con estadísticas generales, gestión de usuarios y administradores, e historial de auditoría de todas las validaciones y accesos realizados.*
- Los Usuarios accederán a un editor web (Monaco) para validar consultas SQL/NoSQL en tiempo real, además de poder integrar la Skill/API o el CLI sqlcheck en sus propios flujos de trabajo.*

4.4 Factibilidad Legal

El desarrollo se basa en librerías bajo licencias Open Source (Express, jsonwebtoken, bcrypt, pg, multer), lo que permite su uso sin costos de regalías.

Además, el tratamiento de credenciales y datos de usuarios se realiza bajo buenas prácticas (contraseñas cifradas con bcrypt, secretos gestionados como variables de entorno), respetando las normativas de protección de datos personales.

Categoría	Implementación en el Proyecto	Marco Legal
Datos Personales	Contraseñas cifradas (bcrypt, 10 rondas) y secretos gestionados mediante variables de entorno (JWT_SECRET, credenciales de base de datos).	Ley N° 29733 (Perú)
Seguridad de Acceso	Control de acceso basado en roles (user/admin/superadmin) y hashing de contraseñas.	Estándar ISO 27001
Licenciamiento	Software desarrollado bajo licencia Open Source (MIT).	Propiedad Intelectual
Confidencialidad	Gestión de secretos (JWT_SECRET, DATABASE_URL, credenciales de administrador) fuera del código fuente, vía variables de entorno.	Protección de Procesos

4.5 Factibilidad Social

El proyecto genera un impacto social positivo en múltiples dimensiones:

- *Fomenta buenas prácticas de desarrollo y calidad de código, al permitir validar consultas SQL/NoSQL antes de ejecutarlas contra una base de datos real, reduciendo errores costosos en producción.*
- *Contribuye al aprendizaje de sintaxis SQL multimotor y MongoDB en estudiantes y desarrolladores junior, mediante mensajes de error claros y sugerencias de corrección.*
- *El proyecto es desarrollado por estudiantes de la Universidad Privada de Tacna, lo que representa un aporte directo del ámbito académico a la comunidad de desarrollo de software.*
- *Reduce la brecha de conocimiento en bases de datos multimotor de equipos pequeños que no cuentan con un especialista dedicado por cada motor SQL.*

El proyecto se alinea principalmente con el ODS 9 (Industria, Innovación e Infraestructura), al fomentar herramientas de calidad de software, y con el ODS 4 (Educación de Calidad), al ser desarrollado como parte de la formación académica de los estudiantes.

4.6 Factibilidad Ambiental

El proyecto tiene un impacto ambiental netamente positivo, alineado con políticas de modernización sostenible:

- *Es un proyecto enteramente digital: la validación de consultas no requiere impresión ni materiales físicos.*
- *La automatización de la validación evita ejecutar consultas de prueba innecesarias directamente contra motores de base de datos reales, optimizando el uso de recursos de cómputo.*
- *El despliegue en Railway con facturación por consumo es energéticamente más eficiente que mantener servidores dedicados encendidos permanentemente, ya que el costo y el consumo escalan según el tráfico real.*
- *La reducción del tiempo de detección de errores de sintaxis mediante validación automatizada evita reprocesos y el consumo adicional de recursos computacionales en ciclos de corrección tardíos.*

5. Análisis Financiero

El análisis financiero del proyecto se enfoca en demostrar la viabilidad económica de la inversión mediante la evaluación de los flujos de caja generados por los ahorros operativos que el Validador de Sintaxis SQL/NoSQL producirá en un equipo de desarrollo de software. Se utilizan tres indicadores financieros estándar: la Relación Beneficio/Costo (B/C), el Valor Actual Neto (VAN) y la Tasa Interna de Retorno (TIR).

5.1 Justificación de la Inversión

5.1.1 Beneficios del Proyecto

Beneficios Tangibles:

- ***Reducción de costos por errores de sintaxis en producción: la detección temprana de errores SQL/NoSQL evita el costo de remediar consultas***

fallidas ya ejecutadas contra una base de datos real, significativamente mayor que corregirlas antes de ejecutar.

- *Optimización del tiempo de revisión de código: al centralizar la validación multimotor (SQL y MongoDB) en una sola herramienta, se reduce el tiempo que el equipo dedica a revisiones manuales dispersas por cada motor.*
- *Aseguramiento de la trazabilidad de las validaciones: el registro automático en audit_logs de cada validación, inicio de sesión y acción administrativa facilita auditorías internas y el seguimiento de incidentes.*

Se proyecta una reducción cercana al 90% de errores de sintaxis que llegan a ejecutarse contra una base de datos real (para los motores cubiertos por el validador) y una reducción estimada del 30%-40% en el tiempo dedicado a revisión manual de consultas multimotor.

Fuente del Beneficio	Impacto Operativo	Ahorro Mensual (S/)	Ahorro Semestral (S/)	Ahorro Anual (S/)
Detección Temprana de Errores de Sintaxis	<i>Evita retrabajo por errores ejecutados en producción en vez de detectados en desarrollo.</i>	<i>S/ 350.00</i>	<i>S/ 2,100.00</i>	<i>S/ 4,200.00</i>
Validación Multimotor Centralizada	<i>Reduce las horas dedicadas a la revisión manual de sintaxis por cada motor SQL.</i>	<i>S/ 250.00</i>	<i>S/ 1,500.00</i>	<i>S/ 3,000.00</i>
Integraciones (Skill/API + CLI + GitHub Action)	<i>Reduce el tiempo de integración de la validación en pipelines y editores externos.</i>	<i>S/ 150.00</i>	<i>S/ 900.00</i>	<i>S/ 1,800.00</i>
TOTAL BENEFICIOS	<i>Ahorro total estimado</i>	<i>S/ 750.00</i>	<i>S/ 4,500.00</i>	<i>S/ 9,000.00</i>

Beneficios Intangibles:

- *Mayor confianza en la calidad de las consultas ejecutadas: los mensajes de error con línea, columna y sugerencia de corrección dan al equipo información objetiva y accionable.*
- *Cultura de validación previa a la ejecución: integrar el análisis en el editor web, la Skill/API y el CLI fomenta la validación como práctica habitual antes de ejecutar cualquier consulta.*

- **Reducción de la carga cognitiva del equipo: al centralizar la validación de múltiples motores SQL y MongoDB en una sola plataforma, se reduce la fricción de usar herramientas dispersas por motor.**

Beneficio Intangible	Descripción del Impacto
<i>Confianza en la calidad de las consultas</i>	<i>Mensajes de error con línea, columna y sugerencia de corrección dan información objetiva y accionable.</i>
<i>Cultura de validación previa</i>	<i>Integración de la validación en el editor web, la Skill/API y el CLI fomenta su uso como práctica habitual.</i>
<i>Reducción de fricción entre herramientas</i>	<i>Centraliza la validación de SQL multimotor y MongoDB en una sola plataforma.</i>
<i>Sostenibilidad operativa</i>	<i>La facturación por consumo en Railway reduce el costo y el consumo de recursos cuando no hay tráfico.</i>
<i>Escalabilidad y oportunidad de integración</i>	<i>La arquitectura (Web + Skill/API + CLI + GitHub Action) permite ofrecer el validador como servicio a otros equipos o integrarlo en más pipelines en el futuro.</i>

5.1.2 Criterios de Inversión

Indicador	Valor Calculado	Interpretación
Inversión Inicial (C)	S/ 15,068.74	Costo total real del proyecto (personal, hardware, infraestructura cloud vía Railway, operativos y generales).
Beneficio Neto Anual	S/ 8,055.00	Ahorro operativo anual proyectado (S/ 9,000 bruto) neto del costo recurrente de infraestructura cloud (S/ 945.00/año).
Relación B/C	B/C = 1.28	Por cada sol invertido, el proyecto recupera S/ 1.28 en valor presente de beneficios (horizonte 3 años, COK 12%). B/C > 1: proyecto económicamente viable.

VAN (COK = 12% anual)	S/ 4,282.31	VAN > 0. El proyecto genera valor positivo real en un horizonte de 3 años.
TIR	TIR ≈ 28%	TIR (28%) > COK (12%). El proyecto supera el costo de oportunidad del capital.

5.1.2.1 Relación Beneficio/Costo (B/C)

Costos (C): Se estima una inversión de S/ 13,103.25 (línea base), que cubre personal de desarrollo, hardware, infraestructura cloud en Railway y costos operativos durante los 3 meses de ejecución.

Beneficios (B): El ahorro anual bruto estimado por la detección temprana de errores de sintaxis y la reducción del tiempo de revisión manual asciende a S/ 9,000.00. Descontando el costo recurrente de infraestructura cloud (S/ 945.00/año), el beneficio neto anual es de S/ 8,055.00.

$$B/C = 1.28$$

5.1.2.2 Valor Actual Neto (VAN)

Este indicador mide el valor actual de los ahorros netos que el Validador de Sintaxis SQL/NoSQL generará durante un horizonte de evaluación de 3 años.

Inversión Inicial: S/ 15,068.74.

Ahorro Neto Mensual Proyectado: S/ 671.25 a partir del mes 4 (lanzamiento del sistema), ya descontado el costo recurrente de infraestructura cloud.

Tasa de Descuento (COK): 12% anual.

Tras el cálculo (horizonte de 3 años, flujos anuales descontados al 12%), el VAN es de S/ 4,282.31. Al ser mayor a cero, se confirma que el proyecto es financieramente rentable y aporta un valor positivo al equipo de desarrollo.

5.1.2.3 Tasa Interna de Retorno (TIR)

Representa la rentabilidad anual de los recursos destinados al desarrollo del motor de validación SQL/NoSQL y sus integraciones (Skill/API, CLI).

TIR Estimada: 28%



Costo de Oportunidad (COK): 12%.

Con una TIR de aproximadamente 28%, superior al costo de oportunidad del capital (12%), se acepta el proyecto. Esto indica que la inversión en el Validador de Sintaxis SQL/NoSQL es más rentable que el costo de oportunidad del capital.

5.1.3 Flujo de Caja

El flujo de caja proyecta los movimientos de dinero del proyecto desde la fase de desarrollo (Meses 0-3) hasta el cierre del horizonte de evaluación de 3 años (36 meses). Las cifras están en soles (S/).

Supuestos del modelo:

- Mes 0 (Pre-desarrollo): adquisición de accesorios menores antes de iniciar.
- Meses 1-3 (Desarrollo): pago de personal, infraestructura cloud activa (Railway) para staging y costos operativos completos.
- Mes 4 (Lanzamiento): fin del desarrollo. El Validador de Sintaxis SQL/NoSQL queda desplegado en producción (Railway) y se materializan los ahorros operativos por detección temprana de errores de sintaxis.
- Meses 4-36: el costo de infraestructura cloud (Railway) es recurrente y variable según consumo.
- El flujo acumulado alcanza su punto de recuperación (payback) proyectado alrededor del mes 25.
- Los costos de infraestructura cloud (Railway) son recurrentes desde el Mes 4 (una vez en producción).
- Los costos operativos de desarrollo (personal, energía, internet dedicado) se reducen al finalizar la Fase 3 (Mes 3).

En base a 3 años

Período	Total Ingresos (S/)	Total Egresos (S/)	Flujo Neto (S/)	Flujo Acumulado (S/)
AÑO 0 (Pre-dev + Desarrollo, meses 0-3)	-	15,068.74	(15,068.74)	(15,068.74)
AÑO 1 (Meses 4-15)	9,000.00	945.00	8,055.00	(7,013.74)
AÑO 2 (Meses 16-27)	9,000.00	945.00	8,055.00	1,041.26
AÑO 3 (Meses 28-36)	6,750.00	708.75	6,041.25	7,082.51



Concepto		Valor (S/)
Inversión Total (Línea Base)		13,103.25
Reserva de Contingencia (10%)		1,310.33
Reserva de Gestión (5%)		655.16
TOTAL PRESUPUESTO DEL PROYECTO		15,068.74
Ahorro Bruto Mensual Proyectado (Mes 4+)		750.00
Costo Infraestructura Cloud/mes (Terraform)		78.75
Ahorro Neto Mensual Proyectado (Mes 4+)		671.25
Tasa de Descuento — COK (anual)		12.00%
Indicador	Valor	Criterio de Decisión
Inversión Inicial (C)	15,068.74	Costo total real (Personal + Hardware + Cloud vía Railway + Operativos + Generales)
Beneficio Neto Anual	8,055.00	Ahorro bruto proyectado: S/ 750/mes × 12 meses, neto del costo de infraestructura cloud
Relación Beneficio/Costo (B/C)	1.28	B/C > 1 → proyecto económicamente viable (horizonte 3 años, COK 12%)
VAN (COK = 12% anual)	4,282.31	VAN > 0 → el proyecto genera valor positivo real
TIR	≈ 28%	TIR (28%) > COK (12%) → se acepta el proyecto
Payback (recuperación)	≈ Mes 25	El flujo acumulado alcanza cero aproximadamente en el mes 25

6. Conclusiones

El estudio de factibilidad confirma que el proyecto de implementación del Validador de Sintaxis SQL/NoSQL es viable en todas sus dimensiones evaluadas:

•Técnicamente, el stack tecnológico seleccionado (Node.js, Express, PostgreSQL, Docker, Railway) es maduro y ya se encuentra desplegado en producción, asegurando una implementación estable. El motor de validación propio



(Lexer/Parser) está cubierto con pruebas unitarias, de mutación y BDD, reduciendo significativamente el riesgo de regresiones.

- *Económicamente, la inversión total asciende a S/ 15,068.74 (línea base S/ 13,103.25 más reservas del 15%). La infraestructura cloud en Railway representa un costo recurrente estimado de S/ 78.75/mes, con facturación por consumo. Bajo esta premisa, el proyecto alcanza un Valor Actual Neto (VAN) de S/ 4,282.31, una Tasa Interna de Retorno (TIR) de aproximadamente 28% y una relación Beneficio/Costo (B/C) de 1.28 en un horizonte de 3 años, consolidándose como una inversión financieramente viable.*

- *Operativamente, el sistema se integra de forma natural en el flujo de trabajo de desarrollo mediante una interfaz web, una Skill/API pública y un CLI, lo que optimiza la detección de errores de sintaxis sin interrumpir las actividades diarias del equipo.*

- *Legalmente, el desarrollo cumple estrictamente con la Ley N° 29733 de Protección de Datos Personales del Perú. La implementación de controles de acceso basados en roles y la gestión de secretos mediante variables de entorno garantizan la seguridad y confidencialidad de la información almacenada.*

- *Social y Ambientalmente, el proyecto promueve una cultura de validación previa y calidad de código, y su despliegue en Railway con facturación por consumo reduce el consumo de recursos frente a servidores permanentemente encendidos. Además, eleva el estándar de formación en bases de datos multimotor de los estudiantes de la Universidad Privada de Tacna.*

La transición de la validación manual por motor a una plataforma centralizada con trazabilidad por validación garantiza un control estricto de los errores detectados y elimina inconsistencias entre herramientas dispersas. La arquitectura del sistema (Web, Skill/API, CLI y GitHub Action), desplegada de forma reproducible, permite su evolución hacia un modelo escalable de servicio para más equipos de desarrollo en el futuro.